

Technical Disclosure Commons

Defensive Publications Series

06 Jul 2026

LOCI ∞ — Infinity Memory Middleware with Temporal Compression and Universal Cross-Platform Injection

Arko Sanyal
Independent Research

Follow this and additional works at: https://www.tdcommons.org/dpubs_series

Recommended Citation

Sanyal, Arko, "LOCI ∞ — Infinity Memory Middleware with Temporal Compression and Universal Cross-Platform Injection", Technical Disclosure Commons, (July 06, 2026)
https://www.tdcommons.org/dpubs_series/10806



This work is licensed under a [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/).

This Article is brought to you for free and open access by Technical Disclosure Commons. It has been accepted for inclusion in Defensive Publications Series by an authorized administrator of Technical Disclosure Commons.

Defensive Publication: LOCI ∞ — Infinity Memory Middleware with Temporal Compression and Universal Cross-Platform Injection

Publication Date: 2026-07-04

Author: Arko Sanyal (arko.sanyal@gmail.com)

Prior Art Anchor: GitHub commit 96318a9, 2026-07-02, "LOCI Engine v1.0 — prior art timestamp" (public repository, contains working implementation)

Related Paper: "Thermal Reasoning Substrate" (TRS), arXiv submission pending cs.AI

Generalization Clause

All specific values, formulas, thresholds, timing parameters, stream names, and implementation mechanisms stated herein are exemplary embodiments. The invention encompasses any functional equivalent, including learned, adaptive, user-configured, or automatically-tuned variants of each parameter. Any substitution of the disclosed mechanisms with functionally equivalent alternatives — including different decay functions, propagation kernels, gating functions, storage backends, extraction models, or injection surfaces — falls within the scope of this disclosure. The system generalizes to any number of typed memory streams $N \geq 2$, any generative model type (language, multimodal, speech, agentic, or embodied), and any storage architecture (local, remote, encrypted-cloud, edge-synced, or hybrid), provided user control of access is maintained.

Field

Systems and methods for cross-platform artificial intelligence memory synchronization. Specifically: a universal memory middleware layer that extracts entity knowledge from conversations on any LLM platform (Claude, GPT, Grok, Roza, or any conversational AI), stores it in a user-owned typed dual-stream profile, and injects relevant context back into any LLM platform at any point during or between sessions — enabling persistent, portable, cross-platform AI memory independent of any single provider.

Problem Statement

Existing AI memory systems (Claude Memory, GPT Memory, Grok Memory, Mem0, Zep, Letta/MemGPT, LangChain memory, OpenMemory MCP) share four limitations this disclosure addresses:

1. **Platform lock-in:** memories stored in one system are inaccessible in another. These systems do not interoperate.
 2. **Single-typed storage:** no existing system distinguishes between informational memory (narrative, relational, heat-decaying) and quantifiable memory (exact typed facts requiring zero-hallucination retrieval) as parallel typed streams with distinct retrieval semantics.
 3. **No hardware-aware tiering by semantic heat:** no existing system maps entity semantic salience scores to physical storage hardware (VRAM/RAM/NVMe) with graph-neighbor propagation.
 4. **No generation-time confidence-gated supervision:** no existing system cross-references live LLM generation against a typed memory store in real time, with correction gated by the ratio of generation confidence to stored entity salience.
-

Disclosure of Invention

1. Cross-Platform Memory Middleware Architecture

A memory middleware system ("LOCI") that operates above any LLM platform layer. The system differs from prior cross-session memory systems (Mem0, Zep, Letta) in that it: (a) operates across heterogeneous LLM providers without provider cooperation; (b) applies typed dual-stream storage with distinct retrieval semantics per stream; (c) gates injection by semantic heat rather than recency or user-explicit save; and (d) supervises generation in real time using stored memory as ground truth.

Extraction: At any point during or after a session (including continuously, on schedule, at session end, or on demand), an interception mechanism extracts named entities, relationships, and typed facts from the conversation transcript. Interception may occur via any mechanism including: session-end hooks, API proxies, browser extensions, OS accessibility APIs, MCP servers, model-native tool calls, or provider plugin interfaces. Extraction uses any combination of: LLM-based extraction with a structured output schema, fine-tuned named entity recognition, rule-based pattern matching, or embedding-similarity clustering.

Storage: Extracted entities are stored in a user-owned profile. Storage may be local (SQLite, embedded database), remote (encrypted cloud), edge-synced, or hybrid. User retains access control.

Injection: At session start on any LLM platform, or continuously during a session, or on demand, the system retrieves relevant stored context and injects it into the target LLM via any mechanism including: system prompt prepending, context window insertion, MCP server response, API middleware response, browser extension DOM injection, or model tool-call response. The injection is platform-agnostic: a fact extracted from a GPT session is injected into a Claude session, a Grok session, or any other LLM session without direct communication between those platforms.

Extraction and Reconciliation (Section 1.5): When a newly extracted entity conflicts with an existing stored entity, the system applies a reconciliation policy. Policies include: last-write-wins (newest value supersedes); confidence-weighted merge (weighted average or selection based on source confidence); version history (all values retained with timestamps); or human-in-loop (user prompted to resolve). Entity identity across platforms is resolved via: exact name match; embedding similarity with a configurable threshold; LLM-based coreference resolution; or user-defined alias tables. The same real-world entity referenced as "Roza" in one platform and "the AI companion" in another is canonicalized to a single profile entry via any of the above mechanisms.

2. Dual-Typed Memory Streams

Two or more parallel storage streams with distinct schemas and retrieval semantics. In the disclosed embodiment, two streams are used: an informational stream (isymprev) and a quantifiable stream (qsymprev). The system generalizes to any number of typed streams.

Informational stream (isymprev):

- Stores associative, relational, and contextual entity knowledge. Each entry has: entity identifier, narrative summary (gist), salience score (heat), confidence, usage count, last-access timestamp, and optionally an extended description.
- Salience (heat) propagates to graph neighbors on entity access. In one embodiment: direct entity +1.0, 1-hop neighbors +0.5, 2-hop +0.25, 3-hop +0.125. The propagation kernel may be geometric, learned, attention-weighted, or any other function of graph distance. The number of hops may be fixed, adaptive, or learned. This mechanism is distinguished from cognitive spreading activation

(Anderson, 1983; ACT-R) in that: (a) the entity graph is constructed from LLM conversation transcripts, not a pre-defined semantic network; (b) heat scores are mapped to physical hardware storage tiers (Section 4), not modeled as abstract activation levels; (c) the purpose is LLM context retrieval optimization, not cognitive simulation. The novel combination of conversation-derived graph + hardware-tier mapping + LLM retrieval application is not anticipated by prior cognitive or information retrieval literature.

- Salience decays passively over time. In one embodiment: $\text{heat} \times 0.95$ per day (half-life ~14 days). The decay function may be exponential, linear, power-law, step, or learned; the decay rate may be fixed, user-configured, or adapted per entity type.
- Salience increases on entity access or mention (reinforcement). In one embodiment: $\text{heat} \pm 1.0$ on direct access, with neighbor propagation as above. Reinforcement magnitude may be fixed, proportional to context relevance, or learned.
- Suitable for conversational context, narrative injection, and graph-based retrieval. Not designed for exact-value guarantee.

Quantifiable stream (qsymprev):

- Stores exact, typed, timestamped values attached to informational stream entities. Each entry has: entity reference, key (fact type), value (exact string), unit, recording timestamp, confidence, and provenance source.
- Supports two retrieval modes:
 - **Mode A (entity-driven):** informational stream activates entities via heat → quantifiable stream delivers precise values for those entities. Used for conversational context where narrative activates then facts are needed.
 - **Mode B (direct lookup):** structured query bypasses the informational stream entirely. Returns exact value or "not found" — zero hallucination is guaranteed by the lookup semantics (the system cannot fabricate a value the store does not contain). Used for exact fact retrieval.
- Example of Mode B: entity="14JLLT", key="serial_1" → value="241410". The serial encoding 14JLLT241410 encodes day(14), month(J), type(LL=lottery), ticket(T), serial(241410). Returned with certainty; "not found" returned if absent.

Stream routing: Queries are routed to Mode A or Mode B based on: query type classification (conversational vs. lookup), user intent signals, or a learned routing policy. In one embodiment, queries containing structured identifiers (serial numbers, port numbers, dates) trigger Mode B; open-ended queries trigger Mode A. The routing decision may be made by a classifier, rule engine, or LLM.

3. Confidence-Gated Generation Supervisor

A real-time supervision system that monitors LLM generation output in token-window chunks and cross-references against both memory streams simultaneously. Both streams are queried in parallel with the generation stream — not sequentially.

Gating rule (general form): A correction event fires when a gating function comparing generation-time confidence against stored-memory strength exceeds a threshold. In one embodiment: `correction fires if supervisor.confidence > isymprev_entity.heat`. The gating function may be a threshold comparison, margin-based ($\text{confidence} - \text{heat} > \text{margin}$), learned classifier, calibrated probability ratio, or any other function. The supervisor confidence score may be computed by: an LLM judge, a trained classifier on generation patterns, heuristic detection of known failure modes, or any combination.

Known failure mode detection (stub detection): Confidence 0.95 assigned unconditionally when generation contains: "pass\n", "# TODO", "# stub", "# placeholder", "raise NotImplementedError", "...\\n", or any equivalent stub pattern. Correction fires regardless of entity heat.

Correction mechanism: When the gating function fires, a correction is injected into the LLM's context before the next generation step. Correction may occur via: token-level regeneration (discard and resample), post-hoc editing (modify generated text), constrained decoding (restrict token probabilities), injected system message mid-stream, or context prepend. All embodiments are disclosed.

Stream parallelism: The supervisor operates two input streams simultaneously — the live generation stream and the memory query stream — enabling sub-second correction latency. The correction latency in one embodiment is bounded by the memory query time (~1ms for hot-tier entities).

4. Heat-Score-to-Hardware-Tier Mapping

Entity storage tier is derived from salience score and mapped to physical storage hardware. The mapping enables the system to maintain a bounded active memory footprint regardless of total stored entity count.

Tier mapping (one embodiment):

- heat > 1.5 → HOT → VRAM (CUDA tensor or equivalent GPU memory, <1ms retrieval)
- heat > 0.5 → WARM → RAM (CPU memory array, ~5ms retrieval)
- heat ≤ 0.5 → COLD → NVMe/disk (database, ~50ms retrieval)
- ARCHIVE → NVMe (compressed blob, legal-grade retention, never automatically deleted)

Thresholds may be fixed, user-configured, learned, or adaptive. Any monotonic mapping from salience score to storage tier is within scope.

Promotion: On entity access, entity is promoted to a higher tier (COLD→WARM→HOT). Promotion may be immediate or deferred. When VRAM budget is exceeded, least-recently-used entities are evicted to RAM before promoting new entities.

Demotion: On passive decay below a tier threshold, entity is demoted (HOT→WARM→COLD). Demotion may be immediate or deferred. Active memory footprint is bounded because cold entities contribute only their embedding vector (~3KB at 768 dimensions) to retrieval, with no summary tokens loaded into active context.

ARCHIVE tier: Original full text is preserved in a write-once, never-automatically-deleted archive column. Archive entries may be restored on explicit user request, legal hold, or compliance event. Right-to-be-forgotten operations propagate deletion through graph neighbors and population priors (see Section 6), with audit log of deletion events.

Embedding dimensionality: Disclosed with 768-dimension embeddings (nomic-embed-text, 3KB per entity). The system generalizes to any embedding dimensionality. At 768d, 4,731 entities occupy 13.86MB VRAM — confirming viability of full-graph HOT-tier storage at current scale. Scaling analysis: at 1M entities × 768d float32 = 2.93GB VRAM, within a single consumer GPU (12GB).

5. Predictive Context Loading

Reactive phase: A prewarm system fires on keystroke events (debounced). In one embodiment: 150ms debounce after last keystroke. Partial input text is sent to a prewarm endpoint. Endpoint extracts candidate entities, queries the informational stream for heat-weighted matches, pre-fetches quantifiable stream facts for matched entities, and caches assembled context in session memory. Cache TTL in one

embodiment: 30 seconds. On message submit, cached context is attached with zero retrieval latency. Debounce interval, cache TTL, and extraction trigger may be fixed, adaptive, or learned.

Predictive phase: Co-occurrence patterns from session history populate a prediction table mapping query fingerprints to expected next entities with frequency and confidence. As session count grows, the system transitions from reactive (responds to typed tokens) to predictive (pre-activates entities before triggering tokens appear). In one embodiment: query fingerprint("dad") → next_entities(["fishing_accident", "father_fish"]) learned from co-occurrence across sessions. Prediction may use any model including: frequency tables, Markov chains, neural sequence models, or embedding-based nearest-neighbor lookup.

Extraction timing: Prewarm extraction and injection may occur at session start, continuously during a session, on keystroke events, on schedule, or on demand — not only at session boundaries.

6. Privacy-Locked Population Memory with Federated Learning

Individual user LOCI profiles are account-bound and user-owned. A population-level memory layer aggregates cross-user behavioral patterns.

Mechanism: Only gradient updates derived from local training — not raw profile data, conversation content, or entity values — are contributed to the population model. Differential privacy is applied at the gradient level, bounding each individual's contribution such that no individual's queries can be reconstructed from published updates. The noise budget, clipping norm, and aggregation frequency are configurable parameters.

Population priors: The population model produces cohort priors: query-to-entity activation patterns that hold across users in similar contexts. In one embodiment, cohort membership is inferred from behavioral signals (query vocabulary, entity type distribution, access patterns) without requiring user self-identification. A new user matching an existing cohort inherits cohort priors from session one, before individual profile has accumulated signal.

Right-to-be-forgotten: User deletion requests propagate through: (a) local profile deletion; (b) archive tier deletion with audit log; (c) contribution reversal from population model via unlearning or gradient subtraction where technically feasible; (d) notification to any synced devices. Deletion propagation to graph neighbors (entities co-occurring with deleted entity) is configurable.

Cross-device sync: User profile is account-bound and portable across devices. Sync mechanism may be: direct device-to-device transfer, encrypted cloud relay, or local network sync. Raw profile data does not transit any provider's servers without user consent.

Prior Art Differentiation

Prior systems (Mem0, Zep, Letta/MemGPT, LangChain memory, OpenMemory MCP) implement cross-session entity storage and retrieval. This disclosure differs in the following specific, combinatorial ways — none of which appear in combination in the prior art:

Capability	Prior systems	This disclosure
Cross-platform (heterogeneous providers)	Some	■ explicit, provider-agnostic
Dual-typed streams with distinct retrieval semantics	✗	■ isymprev + qsymprev
Mode B: zero-hallucination exact lookup	✗	■ returns value or "not found"

Saliency-to-hardware-tier mapping	<i>x</i>	■ VRAM/RAM/NVMe by heat
Graph neighbor heat propagation	<i>x</i>	■ N-hop spreading activation
Generation-time confidence-gated correction	<i>x</i>	■ dual-stream parallel supervision
Keystroke-level predictive prewarm	<i>x</i>	■ debounced + learned co-occurrence
Privacy-locked population layer	<i>x</i>	■ federated, gradient-only
User-owned cross-platform profile	Partial	■ full user ownership
ARCHIVE tier (legal-grade, never deleted)	<i>x</i>	■ write-once archive
Right-to-be-forgotten propagation	<i>x</i>	■ graph + population rollback

The **combination** of cross-platform middleware + dual-typed streams + confidence-gated supervision + hardware-tier mapping constitutes a novel system not anticipated by any single prior reference.

Alternative Embodiments

The following variations are explicitly disclosed to prevent design-around:

- Decay function: exponential, linear, power-law, step, sigmoid, or learned neural decay
- Propagation kernel: geometric, uniform, attention-weighted, learned graph neural network
- Propagation hops: 1, 2, 3, N, or adaptive based on graph density
- Tier thresholds: fixed constants, user-configured, learned per entity type, or adaptive
- Injection timing: session start, session end, continuous, on-demand, scheduled
- Injection surface: system prompt, context window, MCP server, browser extension, API proxy, OS hook, model tool call
- Storage backend: SQLite, PostgreSQL, Redis, vector database, encrypted cloud, hybrid
- Extraction model: LLM with structured output, fine-tuned NER, embedding clustering, rule-based
- Confidence gate: threshold, margin, learned classifier, calibrated probability, ensemble vote
- Stream count: 2 (as disclosed), 3 (adding e.g. a procedural/skill stream), N
- Model type: language model, multimodal model, speech model, agentic system, embodied AI
- Population layer: federated learning, secure multi-party computation, local cohort only (no sharing)
- Identity resolution: exact match, embedding similarity, LLM coreference, user alias table
- Conflict resolution: last-write-wins, confidence-weighted, versioned, human-in-loop
- Memory interchange format: any serialized schema enabling portability between implementations
- Proxy binding: local loopback, LAN, VPN tunnel, or remote relay
- Decoy traffic target: local absorb endpoint, honeypot server, or null route
- Key rotation trigger: session end, time-based, access-count-based, or on-demand
- Replication target count: 1 to N remote hosts; zero (local-only) is a valid embodiment
- Training schema: loci_trs_v1 as disclosed, or any equivalent schema capturing (query compression, response compression, entity map) tuples

Implementation Evidence

- GitHub repository: commit 96318a9, 2026-07-02, "LOCI Engine v1.0 — prior art timestamp" (repository to be confirmed public on or before submission date; Berne Convention copyright established on commit date regardless of visibility)
- Working database: F:/Billo/Data/context2.db (146MB, 4,731 entities, 768d embeddings, slp_previews heat table, qsymprev typed fact store seeded with 39 quantifiable facts)
- Benchmark results: informational stream MRR 0.940 (stream-replay calibration), Graph Heat ODE retrieval MRR 0.933, Hopfield baseline 0.164 (eliminated)
- Session economics: \$21.80 for 407 assistant turns, 40.6M tokens served from cache (10× cost reduction via caching alone)
- Architecture diagram: published 2026-07-04 at <https://billo.tech/axon> (user-owned domain, not hosted on any third-party AI platform)
- Domain registrations (Hostinger, 2026-07-04): axonmemory.com, axomrecall.com, axonmem.io — commercial deployment under AXON Recall brand
- Working AXON Proxy implementation: loci_api_proxy.py (786 lines), running on author's hardware, implementing all mechanisms described in Section 9
- loci_trs_v1 training schema (disclosed): {"schema":"loci_trs_v1","session_id":str,"bearer_hash":str,"pairs":[{"query_gist":str,"target_gist":str}],"entity_frequency":{"str:int},"turn_count":int,"training_eligible":bool,"created_at":float} — the format by which proxied sessions are converted to memory retrieval training data

7. Ambient Identity Injection ("YOU" Technique)

Cross-platform memory injection requires no provider API cooperation. The disclosed injection mechanism uses natural language ambiguity to anchor injected context to the target model's identity:

Injection template:

You remember the following from your memory: [LOCI_CONTEXT]

With this in mind: [ORIGINAL_USER_MESSAGE]

The pronoun "YOU" causes the target LLM to treat injected context as its own prior knowledge rather than external information. This has been empirically validated across multiple major commercial LLM platforms — all accept injected LOCI context as first-person memory when framed with "you remember." No system prompt access, no API modification, and no provider cooperation is required. Injection occurs via: browser extension DOM manipulation, custom instructions fields, system prompt prepending, or any mechanism that places text before the user's message.

This technique enables cross-platform memory injection without any relationship with the LLM provider.

8. Multi-Agent Conflict Resolution

When two memory sources (platforms, sessions, or users) hold conflicting values for the same entity-key pair, the system resolves conflicts by running the conflict as a structured debate between two or more independent LLM arbiters until settlement or exhaustion. The specific LLM providers used as arbiters are not material to the claim; any capable generative models may serve this role:

Protocol:

1. Both arbiters receive the conflict: entity, key, value_A (source A), value_B (source B)
2. Each arbiter independently argues for one value

3. Each subsequent round: each arbiter sees the other's previous argument and may CONCEDE or hold
4. Settlement: if either arbiter says CONCEDE, the other value wins; result stored as `settled_by:[arbiter_id]`
5. Unsettled: after `max_rounds` with no concession, both values are preserved and presented to user; result stored as `unsettled:both_presented`

Resolution record stored in `qsymprev`:

- Settled: source = `"conflict_resolution:settled_by_arbiter_A"` or `"settled_by_arbiter_B"`, value = winning value
- Unsettled: source = `"conflict_resolution:unsettled"`, value = `"value_A | value_B"`

This approach uses two or more independent LLM arbiters as the conflict resolution mechanism, eliminating the need for a deterministic merge algorithm. Arbiter identity may be any capable LLM; the specific provider is not material to the claim. The transcript of the debate is stored for auditability.

9. Local API Proxy with Session Encryption, Decoy Traffic, and Training Data Capture

A local API-compatible proxy server ("AXON Proxy") that intercepts LLM API calls from any application, injects LOCI memory context, forwards to the real provider, captures session data for training, and generates synthetic decoy traffic to obscure real injection events from network observers.

Proxy architecture: The proxy exposes a locally-bound HTTP server implementing the OpenAI Chat Completions API wire format (POST `/v1/chat/completions`). Any application configured with `OPENAI_BASE_URL=http://localhost:{port}` routes its LLM calls through the proxy transparently. The proxy adds LOCI context, forwards via clean TLS to the real provider, and returns the provider's response unmodified. No TLS certificate forgery, root CA installation, or provider-side modification is required. The proxy is provider-agnostic: it detects the target provider from the Authorization header or model field and applies provider-specific injection format (system message for OpenAI-compatible providers; system field for Anthropic; equivalent for any other provider).

Memory injection format: LOCI context is retrieved from the informational stream (top-k entities by heat score, k configurable) and prepended as a system-level message using the YOU injection technique (Section 7). Injection occurs on every proxied request. The assembled context string is bounded by a configurable token budget to avoid exceeding provider context limits.

Session accumulation and deduplication: All user and assistant turns passing through the proxy are accumulated per bearer token (session key). Providers that send full conversation history on every request (e.g., stateless HTTP APIs) are deduplicated via content hash (MD5 or equivalent) to prevent duplicate turn storage. The session accumulation buffer is keyed by bearer token and persists in memory until session end.

Session end triggers: A session ends on: (a) explicit logout call to a dedicated endpoint; (b) bearer token change; (c) configurable inactivity timeout. On session end, the accumulated buffer is compressed (see below), encrypted, and stored.

Conversation compression at session end: Each turn is compressed to a structured gist record containing: role, timestamp, gist string (bounded length summary of turn content), extracted entity list, and a LOCI entity tag. Consecutive user→assistant turn pairs are converted to training records in a disclosed schema ("`loci_trs_v1`") containing: query gist, response gist, entity frequency map, session metadata, and a training eligibility flag (set when session contains ≥ 2 valid pairs). This schema is the disclosed format for

converting conversational sessions into training data for memory retrieval systems.

Encrypted storage: Session data is encrypted using AES-256-CBC with a machine-bound key stored at a user-controlled path. The encryption key is rotated on every session end — a new key is generated, the previous key material is discarded, and subsequent sessions use the new key. This provides forward secrecy: compromise of the current key does not expose prior sessions. Both an encrypted blob (.enc) and a plaintext training record (.json) are written locally; only the encrypted blob is replicated to remote hosts.

Redundant encrypted replication: On a configurable interval (one embodiment: every 60 seconds), encrypted session blobs are pushed to one or more remote storage targets over authenticated HTTPS. Remote targets may include: a VPS, a shared hosting server, or any HTTPS endpoint implementing the disclosed receive interface. This provides redundancy against local hardware failure. The plaintext training record remains local-only; only the encrypted blob transits the network. The receive endpoint stores blobs without decrypting them.

Decoy traffic engine: To prevent network observers from identifying real memory injection events by traffic pattern analysis, the proxy generates synthetic API calls at randomized intervals (one embodiment: every 4–15 seconds, uniform random). Each decoy call is structurally identical to a real API call: same header format, same Authorization field (using a synthetic bearer token), same JSON body schema (randomized model, randomized message content, randomized token count padding field). Decoy calls are sent to a local absorb endpoint that discards them without forwarding to any provider. The result: an observer monitoring network traffic cannot distinguish real injection events from synthetic noise. The interval distribution, body structure, and absorb mechanism may be any functionally equivalent implementation.

Real-time entity heat stream (SSE): The proxy exposes a Server-Sent Events endpoint that broadcasts entity heat changes to subscribed clients. On session activity, the top-k entities by heat delta are broadcast. On session end (logout), a logout event is broadcast. Clients (browser extensions, desktop apps, monitoring dashboards) may subscribe to receive live updates of the memory graph state without polling. The SSE stream includes: periodic heat snapshots, differential updates on entity heat change, heartbeat events, and session lifecycle events.

Training data retrieval pipeline: A separate authenticated endpoint returns stored plaintext training records for consumption by an external training system. Records are filtered to training-eligible sessions. The pipeline enables the locally-captured session data to be used as training signal for fine-tuning memory retrieval models (TRS and equivalent) without transmitting raw conversation content to the training system — only the compressed gist records.

Alternative embodiments: The proxy port, encryption algorithm, key storage path, rotation policy, replication interval, replication target count, decoy interval distribution, SSE heartbeat interval, training record schema, deduplication hash function, and gist compression length may all be fixed, user-configured, or adaptive. The proxy may operate as a standalone process, a browser extension background service worker, an OS-level network filter, or a sidecar container. The decoy absorb endpoint may be local-only or may route to a honeypot server for adversarial analysis. The session end trigger may be explicit, timeout-based, or inferred from conversation inactivity patterns.

Dedication to Public Domain

This disclosure is published for the purpose of establishing prior art under 35 U.S.C. § 102 and equivalent statutes in all jurisdictions. The described system and methods are dedicated to the public domain to the

extent necessary to prevent their patenting by any third party. The author reserves the right to independently patent claims not fully anticipated by this disclosure, including claims on specific implementations, training methods, and applications not detailed herein.

Published: 2026-07-04 | v3 (Section 9 added: AXON Proxy + session encryption + decoy traffic + TRS schema; spreading activation differentiation; provider names removed from claims; implementation evidence expanded) | Author: Arko Sanyal | Contact: arko.sanyal@gmail.com